APTC101

Software para arriendo de vehículos Car-REnt

Nombre Juan Diaz Palma

Fecha 20 de Octubre 2026

Índice de contenidos

1. Introducción *3*

1.1 Sobre el caso **3**

1.2 Objetivo general **3**

1.3 Objetivos específicos **3**

2. Descripción del problema *4*

2.1 Datos y reglas de cada elemento **4**

2.2 Funcionalidad requerida **4**

2.3 Requisitos de programación y de prueba **4**

3. Análisis y diseño de la solución *5*

3.1 Diagrama de clases **5**

3.2 Identificación de clases y atributos **7**

3.3 Asignación de comportamientos (métodos) **7**

3.4 Relaciones, tipos y cardinalidades **8**

3.4.1 Por qué no se usó composición 8

3.4.2 Navegabilidad 8

3.5 Buenas prácticas y convenciones de POO aplicadas **8**

3.6 Implementación y resultados de las pruebas **9**

4. Conclusiones *10*

4.1 Dificultades y análisis crítico **10**

4.2 Nivel de logro de los objetivos **10**

5. Bibliografía *11*

1. Introducción

1.1 Sobre el caso

La empresa de arriendo de vehículos Car-REnt necesita un sistema informático que administre la información de sus automóviles, clientes, arriendos y devoluciones. Hoy esa gestión es manual y es propensa a errores, arrendar un auto que ya está arrendado, arrendar a un cliente que ya no está vigente o registrar una devolución con datos que no corresponden. La motivación de este trabajo es resolver ese problema aplicando los fundamentos de la ***Programación Orientada a Objetos (POO)***, de modo que las reglas del negocio queden dentro de las propias clases y no repartidas por el programa.

1.2 Objetivo general

Se realizará el diseño y construcción para dar a una solución para Car-REnt que parta de un diagrama de clases UML y termine en un programa Java que funcione y se pruebe según lo pedido en el enunciado.

1.3 Objetivos específicos

- Identificar las clases, atributos y métodos del caso, y asignar cada comportamiento a la clase que corresponde.
- Modelar las relaciones entre clases (asociación y agregación), justificando tipos y cardinalidades.
- Programar las clases en Java aplicando encapsulamiento, constructores, mutadores con validación, accesores y toString.
- Implementar las operaciones del negocio: deshabilitar cliente, asignar mantención, evaluar e ingresar arriendo, obtener monto, generar ticket e ingresar devolución.
- Probar el sistema con una clase de prueba (método main) que cubra los casos pedidos.

2. Descripción del problema

Car-REnt contrata el diseño y programación de un sistema que administre y almacene la información de sus automóviles, clientes, arriendos y devoluciones, usando POO y sus buenas prácticas. El enunciado se resume a continuación.

2.1 Datos y reglas de cada elemento

Elemento	Características y reglas
Vehículo	Patente (largo 8), marca, modelo, año de fabricación (desde 2000 hasta el año actual) y condición (D disponible, A arrendado, M en mantención). Al ingresar un vehículo su condición inicial es D y patente, marca y modelo se reciben en mayúscula.
Cliente	Cédula (largo 10, penúltimo carácter un guion, último un dígito 0-9 o k), nombre y vigencia (true vigente, false no vigente). Al ingresar un cliente su vigencia inicial es true.
Arriendo	Número de arriendo, vehículo, cliente, fecha de arriendo y días arrendados (mayor que 1 y menor que 10). El monto se obtiene del precio diario que el usuario ingresa al ingresar el arriendo.
Devolución	Pertenece a un arriendo específico y tiene una fecha de devolución que no puede ser menor a la fecha del arriendo.

2.2 Funcionalidad requerida

Letra	Operación	Descripción resumida
a	Deshabilitar cliente	El gerente deja al cliente no vigente para que no pueda arrendar.
b	Asignar vehículo en mantención	Deja la condición en M. No es posible si el vehículo está arrendado.
c	Evaluar arriendo	Valida que el cliente esté vigente y que el vehículo tenga condición D. Retorna true o false.
d	Ingresar arriendo	Ejecuta la evaluación (c). Si es exitosa, deja el auto en A y retorna true; si no, no guarda el arriendo y retorna false.
e	Obtener monto a pagar	Días de arriendo multiplicados por el precio diario.
f	Generar ticket de arriendo	Muestra los datos del arriendo con el formato del ejemplo, usando el monto de (e).
g	Ingresar devolución	Valida que los datos del vehículo devuelto correspondan a los del arriendo. Si son correctos, deja el auto en D y retorna true; si no, retorna false.

2.3 Requisitos de programación y de prueba

- Las validaciones de atributos se programan en métodos públicos de cada clase (mutadores) y se ejecutan al asignar los valores, incluso al instanciar los objetos.
- Cada clase debe tener un único método que reciba un mensaje (String) y lo muestre al usuario.
- Se programa un main en una clase de prueba que, con datos directos (sin teclado), cree y valide un cliente y un vehículo, deshabilite al cliente, asigne el vehículo a mantención, ingrese dos arriendos (uno exitoso con ticket y otro fallido, con objetos distintos a los anteriores) y dos devoluciones (una exitosa y otra fallida), mostrando el estado de los objetos y mensajes al usuario.

3. Análisis y diseño de la solución

Siguiendo la recomendación de la guía (diseñar primero, programar después), el análisis se hizo sobre el enunciado: los sustantivos con datos propios se transformaron en clases, los datos en atributos y los verbos de la funcionalidad en métodos. Se muestra el diagrama de clases y, desde la página siguiente, se justifica cada decisión.

3.1 Diagrama de clases

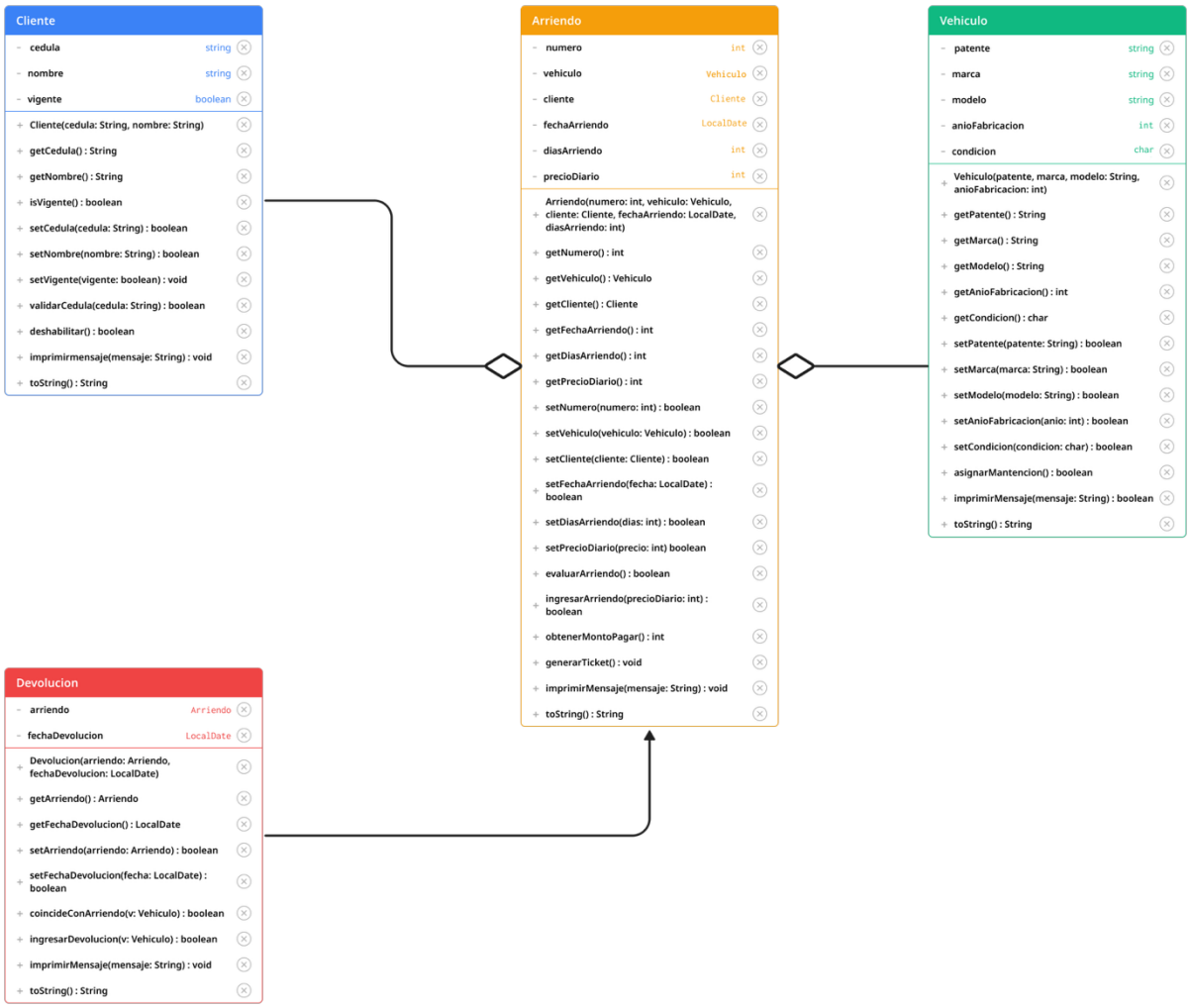


*Figura 1. Diagrama de clases UML del sistema Car-REnt: cuatro clases, **dos agregaciones y una asociación**. El rombo indica agregación y la flecha asociación navegable.*

3.2 Diagrama de clases

Considerando el sistema diseñado, se realiza un diagrama UML de caso de uso donde se explica el manejo de la aplicación tanto por el empleado como por los usuarios.

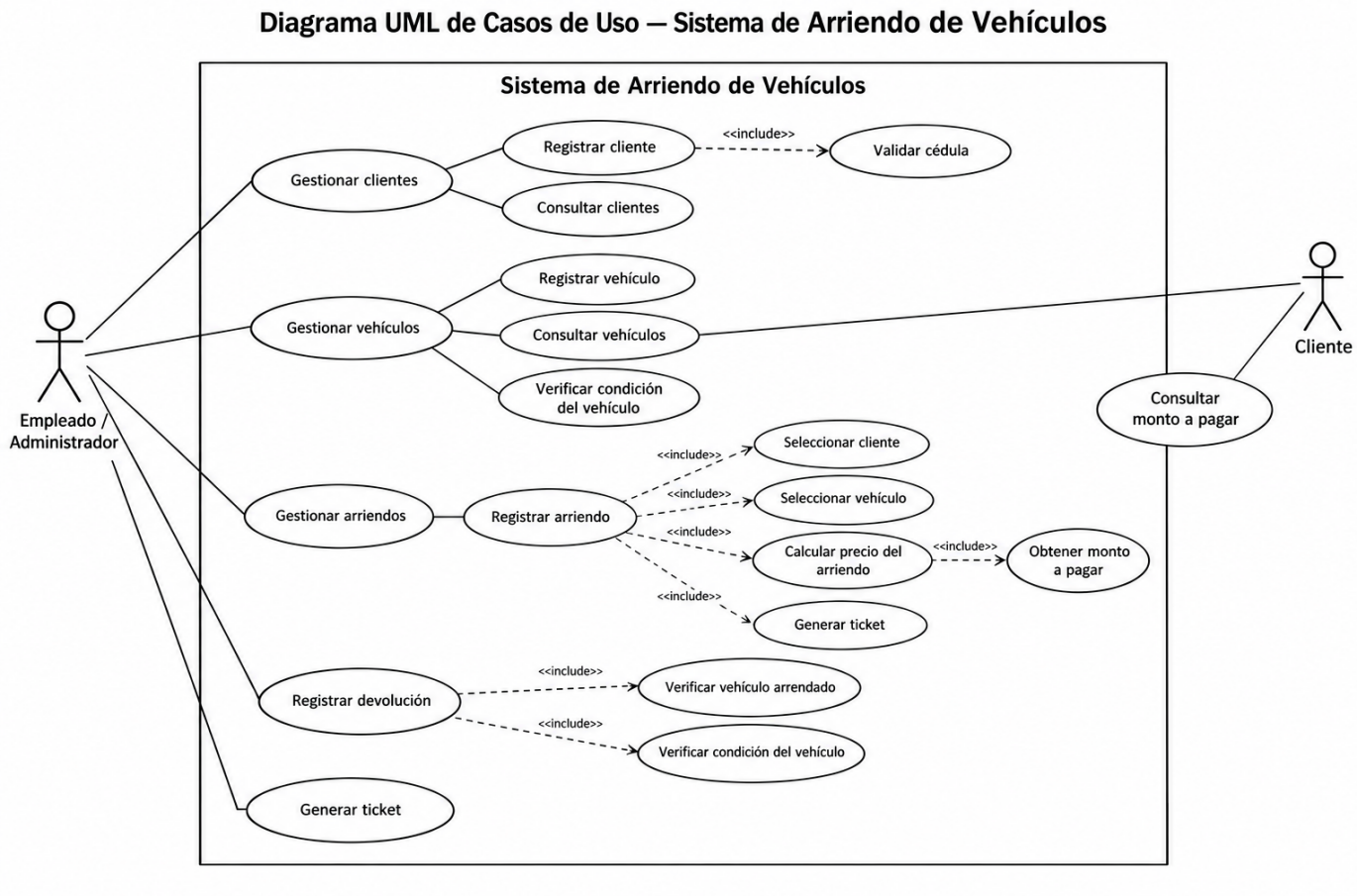


*Figura 2. Diagrama UML de Caso de Uso donde se presentan tanto los usuarios **clientes** como **empleados**.*

3.3 Identificación de clases y atributos

Se identificaron cuatro clases. **No se creó una clase para el gerente porque es quien ejecuta algunas operaciones, pero no tiene datos que el sistema deba guardar**. Tampoco se creó una clase para la empresa, ya que el enunciado no pide administrar colecciones; se deja como mejora futura.

Clase	Atributos (privados)	Justificación y reglas
Vehiculo	patente: String marca: String modelo: String anioFabricacion: int condicion: char	Son las características pedidas. La condición es un solo carácter (D, A, M), por lo que char es suficiente y simple. Patente, marca y modelo se guardan en mayúscula al setearlos. La condición parte en D dentro del constructor.
Cliente	cedula: String nombre: String vigente: boolean	La cédula es String porque incluye guion y puede terminar en k. La vigencia es boolean (true o false) y parte en true.
Arriendo	numero: int vehiculo: Vehiculo cliente: Cliente fechaArriendo: LocalDate diasArriendo: int precioDiario: int	El vehículo y el cliente son atributos de tipo clase, porque expresan las relaciones. La fecha es LocalDate para poder compararla con la de devolución. El monto no es atributo: se calcula (días por precio diario) en un método, así nunca queda inconsistente con sus datos. El precio diario es atributo porque el usuario lo entrega al ingresar.
Devolucion	arriendo: Arriendo fechaDevolucion: LocalDate	Pertenece a un arriendo específico (atributo de tipo Arriendo). Con ese arriendo puede validar su fecha y comparar el vehículo devuelto con el arrendado.

3.4 Asignación de comportamientos (métodos)

Cada operación se asignó a la clase que posee los datos que necesita. Todos los métodos son pequeños y retornan un valor específico (boolean o int) cuando corresponde.

Clase	Método	Por qué está en esta clase
Cliente	deshabilitar(): boolean	Solo cambia la vigencia del propio cliente (operación a).
Cliente	validarCedula(String): boolean	La regla del formato de cédula pertenece al dato cédula; la usa setCedula.
Vehiculo	asignarMantencion(): boolean	Depende solo de la condición del propio vehículo; si está en A no puede pasar a M (operación b).
Arriendo	evaluarArriendo(): boolean	Necesita cliente y vehículo, que el arriendo conoce (operación c).
Arriendo	ingresarArriendo(int precioDiario): boolean	Recibe el precio, invoca evaluarArriendo, y si resulta deja el vehículo en A. Retorna true o false (operación d).
Arriendo	obtenerMontoPagar(): int	Usa días y precio diario, que son datos del arriendo (operación e).
Arriendo	generarTicket(): void	Usa los datos del arriendo, del cliente y del vehículo, e invoca obtenerMontoPagar (operación f).
Devolucion	ingresarDevolucion(Vehiculo): boolean	Compara el vehículo recibido con el del arriendo; si coincide lo deja en D (operación g).

Clase	Método	Por qué está en esta clase
Devolucion	coincideConArriendo(Vehiculo): boolean	Separa la comparación de datos, para que ingresarDevolucion sea más corto y claro.
Todas	setX(valor): boolean	Mutadores públicos con la validación del atributo. Retornan true si asignaron el valor.
Todas	getX() / isX()	Accesores: permiten leer los atributos privados.
Todas	imprimirMensaje(String): void	Concentra la salida de mensajes de la clase en un solo método (requisito adicional b).
Todas	toString(): String	Entrega el estado del objeto para mostrarlo en las pruebas.

3.5 Relaciones, tipos y cardinalidades

Relación	Tipo	Cardinalidad	Justificación
Arriendo - Cliente	Agregación (rombo en Arriendo)	Arriendo 0..* - 1 Cliente	Un arriendo tiene exactamente un cliente (1). Un cliente puede tener muchos arriendos a lo largo del tiempo, o ninguno si recién se ingresó (0..*). El cliente existe antes del arriendo y sigue existiendo después, por eso es agregación.
Arriendo - Vehiculo	Agregación (rombo en Arriendo)	Arriendo 0..* - 1 Vehiculo	Un arriendo es de un solo vehículo (1). Un vehículo participa en muchos arriendos en el tiempo, uno a la vez (0..*). El vehículo es parte de la flota y no desaparece al terminar el arriendo.
Devolucion - Arriendo	Asociación navegable (flecha hacia Arriendo)	Devolucion 0..1 - 1 Arriendo	Cada devolución pertenece a un solo arriendo (1). Un arriendo tiene a lo más una devolución, y ninguna mientras el auto no se devuelve (0..1). Es una asociación porque no hay relación todo-parte. La flecha indica que Devolucion conoce a su Arriendo, pero Arriendo no necesita conocer su devolución (menor acoplamiento).

3.5.1 Por qué no se usó composición

En una composición la parte nace y muere con el todo. Si se elimina un arriendo, no deben desaparecer el cliente ni el vehículo, **porque son objetos que ya existían y que se reutilizan**. Por eso en el programa el arriendo recibe objetos ya creados (agregación) y no los crea por dentro. La devolución tampoco es una parte física del arriendo, sino un evento posterior que hace referencia a él.

3.5.2 Navegabilidad

Arriendo conoce a Cliente y Vehículo, y Devolución conoce a Arriendo. Las flechas van solo en un sentido, lo que evita dependencias circulares entre clases.

3.6 Buenas prácticas y convenciones de POO aplicadas

- **Encapsulamiento:** todos los atributos son privados y se acceden con **get y set**.

- **Validación en los mutadores:** cada set valida y retorna boolean. Los constructores usan los mismos set, de modo que los datos de instanciación también quedan validados sin repetir código.
- **Métodos atómicos con retornos específicos:** cada método hace una sola cosa (por ejemplo, *ingresarArriendo* evalúa, actualiza la condición y retorna *true* o *false*).
- **Mensajes centralizados:** cada clase tiene un único imprimirMensaje; el resto del código no escribe en pantalla.
- **Convenciones de nombres:** clases en PascalCase, atributos y métodos en camelCase, sin tildes ni eñes en los identificadores.
- **Claridad:** código con comentarios, bien indentado, y con la clase de prueba separada de las clases del negocio.

3.7 Implementación y resultados de las pruebas

La clase PruebaCarRent ejecuta los casos pedidos con datos directos. La tabla resume qué se probó y el resultado obtenido; luego se muestra la salida real del programa.

Caso	Qué se probó	Resultado obtenido
a	Cliente con cédula y nombre inválidos; luego corregidos.	Se rechazaron ambos datos con mensaje; tras corregir, el cliente quedó vigente (true).
b	Vehículo con patente y año inválidos; luego corregidos.	Se rechazaron con mensaje; marca, modelo y patente quedaron en mayúscula y la condición en D.
c	Deshabilitar al cliente.	El cliente quedó con vigente=false.
d	Asignar el vehículo a mantención.	La condición pasó a M.
e	Arriendo 1 (correcto) y arriendo 2 (vehículo ya arrendado).	El 1 se ingresó, se generó el ticket (3 días x $45000 = $135000) y el auto quedó en A. El 2 se rechazó con mensaje; no hay ticket.
e (extra)	Mantención a un vehículo arrendado.	Se rechazó con mensaje; la condición se mantiene en A.
f	Devolución 1 (correcta) y devolución 2 (vehículo distinto).	La 1 se ingresó y el auto volvió a D. La 2 se rechazó; el otro vehículo no cambió. Además se probó que la fecha anterior al arriendo se rechaza.

4. Conclusiones

Se diseñó un diagrama de clases con cuatro clases, sus atributos, sus métodos y tres relaciones (dos agregaciones y una asociación), y se programó en Java tal como se diseñó y solicitó. Las siete operaciones del enunciado (deshabilitar cliente, asignar mantención, evaluar arriendo, ingresar arriendo, obtener monto, generar ticket e ingresar devolución) funcionan, las validaciones se ejecutan en los mutadores y los mensajes salen por el método *imprimirMensaje* de cada clase. La clase de prueba mostró los casos exitosos y los fallidos y el estado de los objetos fue el esperado. Diseñar primero redujo los cambios durante la programación.

4.1 Dificultades y análisis crítico

- Elegir el tipo de relación entre Devolucion y Arriendo: se evaluó composición y agregación, y se eligió asociación porque no es una relación todo-parte. Es una decisión discutible y quedó argumentada.
- Interpretar la regla de los días (mayor que 1 y menor que 10): se tomó como valores de 2 a 9.
- Cuando un dato inválido llega al constructor, el atributo queda sin valor (null o 0) hasta que se corrige con el set. Funciona para la prueba, pero en un sistema real sería mejor impedir la creación del objeto, por ejemplo con excepciones.
- El enunciado pide que el sistema almacene la información, pero solo define operaciones. Por eso ingresar un arriendo cambia la condición del auto y retorna true, pero los arriendos no se guardan en una colección.
- La condición del vehículo como char es simple, pero permite valores incorrectos si no se valida; por eso setCondicion solo acepta D, A o M.

4.2 Nivel de logro de los objetivos

Se identificaron las clases y métodos, se modelaron y justificaron las relaciones, se programó con encapsulamiento y validaciones, se implementaron todas las operaciones y se probó el sistema. El objetivo general se considera logrado en el alcance pedido; lo que no se cubre es el almacenamiento de datos y la interacción con el usuario, que el enunciado no exigía.

5. Bibliografía

UNAB Online. (s. f.). *APTC101 - Semana 3. Sumativa 1: Guía de aprendizaje 1* [Guía, instrucciones y rúbrica del curso]. Universidad Andrés Bello.

Apuntes y material del curso APTC101 sobre programación orientada a objetos y diagramas de clases UML. Universidad Andrés Bello.

Booch, G., Rumbaugh, J. y Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2.ª ed.). Addison-Wesley.

Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3.ª ed.). Addison-Wesley.

Oracle. (s. f.). *The Java Tutorials: Learning the Java Language*. https://docs.oracle.com/javase/tutorial/java/index.html

Oracle. (s. f.). *Java Platform, Standard Edition 21 API Specification: java.time.LocalDate*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDate.html